Final Report — LLM-guided LLVM IR Mutation for Differential Compiler Testing

Title: Can LLMs Generate Valid LLVM IR Test Cases for Differential Compiler Testing?

Abstract This project implements and evaluates a prototype workflow that uses LLMs to generate or mutate LLVM IR test cases, applies strict verification and filtering, and compares outputs with grammar- and random-based mutation strategies. The goal is to determine whether LLM-generated IR can provide useful, novel test cases for differential compiler testing beyond existing fuzzers.

1. Problem Statement

- See the project problem statement: the central question is whether LLMs can produce semantically-interesting, valid LLVM IR for differential testing instead of only producing invalid or low-value outputs. (See docs/PROBLEM_STATEMENT.md for full text.)

2. Objectives

- Survey compiler fuzzing methods (Csmith, YARPGen, coverage-guided fuzzers).
- Enumerate LLVM IR validity constraints.
- Build a prototype LLM-guided mutation + filtering workflow.
- Evaluate LLM vs grammar/random mutation on validity, novelty, and detection power.

3. System Design & Implementation

3.1 Architecture

- The backend implements a generation → deduplicate → validate → manifest pipeline. Mutator types include `LLMMutator`, `GrammarMutator`, and `RandomMutator`. Generated mutants are logged to `backend/data/logs/raw_mutants.json` and validation results are stored in `backend/data/logs/validity_logs.json`. See the full backend flow in docs/BACKEND_WORKFLOW.md.

3.2 Key Components

- Mutator implementations: located under `backend/api/app/` (LLM, grammar, random handlers).
- Validation: `llvm-as` followed by `opt -passes=verify` in isolated workers; valid output moved to `backend/data/valid_mutants/`.
- Manifest and analysis endpoints aggregate runs for comparison and export.

4. Methodology

4.1 Generation

- Seeds (IR files) are provided via the frontend upload. Each seed is mutated using one of the mutators; outputs are deduplicated before write.

4.2 Validation and Filtering

- Each mutant is parsed with `llvm-as` and verified with `opt -passes=verify`. Invalid mutants are recorded under `invalid_mutants/` with taxonomy tags (broken_ssa, type_error, invalid_phi, etc.). Optional semantic-triviality checks are applied to valid mutants.

4.3 Evaluation

- Metrics: validity rate (fraction passing verification), mismatch rate (differential behavior between optimization levels), and semantic diversity (qualitative analysis of code patterns exercised). Study runs and aggregated metrics are stored in the logs and manifest (see DOCUMENTATION_INDEX.md).

5. Experiments & Results

- Phase 3 execution produced recorded artifacts and metrics (see `DOCUMENTATION_INDEX.md`): test pass rate for unit tests is 100% and a recorded `111 mutants` in the Phase 3 validation snapshot.
- Observed patterns:
  - Grammar-based mutation yields higher validity rates and more syntactically-correct mutants.
  - LLM-based mutation can produce novel, non-trivial IR patterns but at a higher invalidity rate and greater variance; many LLM outputs require sanitizer and normalization steps.

6. Failure Modes and Lessons Learned

- Common failure classes: broken SSA, type errors, invalid PHI nodes, missing `@main` or required declarations, and semantically-trivial changes that do not affect optimizer paths.
- Practical lessons:
  - Deduplication and early hash checks reduce wasted validation work.
  - Isolated validation workers prevent toxic subprocess effects.
  - Semantic-triviality checks are needed to filter valid-but-useless mutants.

7. Artifacts, Figures, and Evidence

- Generated logs and manifests: `backend/data/logs/` (raw_mutants.json, validity_logs.json, manifest.json).
- Visual knowledge graph produced by the `graphify` run is available at: graphify-out/graph.html.

8. Conclusion

- LLMs are a promising complementary source of IR mutations — they can introduce patterns not captured by deterministic grammar rules — but they cannot replace grammar/random fuzzers without strong validation and filtering. For coursework deliverables, the current prototype and documentation provide the basis for a clear presentation: implemented workflow, taxonomy of failure modes, and a comparative study plan.

9. Recommendations / Future Work

- Implement semantic-equivalence/triviality detection to flag valid-but-uninteresting mutants.
- Run controlled studies over a broader and diverse seed set (arithmetic, memory, calls, control-flow heavy examples).
- Add metrics for semantic diversity (coverage-based or optimizer-path-based) to quantify novelty.

10. References

- Project problem statement: docs/PROBLEM_STATEMENT.md
- Backend workflow: docs/BACKEND_WORKFLOW.md
- Documentation index & Phase metrics: docs/DOCUMENTATION_INDEX.md

Appendix: Where to find code and logs

- Backend code: `backend/api/app/` (mutators, services, routes)
- Logs & mutants: `backend/data/logs/`, `backend/data/valid_mutants/`, `backend/data/invalid_mutants/`